BÁO CÁO BÀI 1

Trao đổi dữ liệu sử dụng HTTP API

Phạm vi: Phần 1 - Bài lab trên lớp (theo nhóm).

Nhóm: ..... Lớp: .....
Thành viên: .....

1. Mục tiêu và phương án thực hiện

Xây dựng server có thông tin nhiệt độ và độ ẩm. Client sử dụng HTTP request với phương thức GET để nhận thông tin và hiển thị kết quả. Chương trình sử dụng Python và thư viện chuẩn, không cần cài thêm gói phụ thuộc.

Server cung cấp dữ liệu mẫu cố định **23°C** và **33%** theo ví dụ trong đề. Dữ liệu truyền qua HTTP thật trên máy cục bộ; không sử dụng cảm biến vật lý.

Thành phần	Chức năng
server.py	Lắng nghe tại 127.0.0.1:8000; trả dữ liệu JSON.
client.py	Gửi GET, phân tích JSON, hiển thị nhiệt độ và độ ẩm.
client.html	Client trình duyệt, hiển thị trực quan và phản hồi HTTP.

2. Luồng trao đổi và định dạng dữ liệu

```
Client -> GET /api/sensor -> Server
Client <- 200 OK + JSON   <- Server

{"temperature": 23, "humidity": 33}
```

temperature: nhiệt độ, đơn vị °C. **humidity:** độ ẩm, đơn vị %. Header phản hồi: **Content-Type: application/json; charset=utf-8**.

3. Cách chạy

Mở hai terminal tại thư mục chứa mã nguồn. Ở terminal thứ nhất chạy server, giữ server hoạt động rồi chạy client ở terminal thứ hai.

```
python server.py
python client.py
```

Để dùng client web, mở **http://127.0.0.1:8000/** và bấm **Gửi GET tới server**. Nhấn Ctrl+C ở terminal server để dừng.

4. Mã nguồn chính của server

File server.py. Đoạn trích dưới đây là nhánh trả dữ liệu cảm biến. Mã nguồn đầy đủ trong file còn phục vụ các trang HTML và cho phép cấu hình host/cổng.

```
import argparse
import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from urllib.parse import urlsplit

ROOT = Path(__file__).resolve().parent
SENSOR_DATA = {"temperature": 23, "humidity": 33}


class SensorHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        path = urlsplit(self.path).path
        if path == "/api/sensor":
            # Chuyen du lieu Python thanh JSON va gui ve client.
            body = json.dumps(SENSOR_DATA).encode("utf-8")
            content_type = "application/json; charset=utf-8"
        else:
            self.send_error(404, "Endpoint not found")
            return

        self.send_response(200)
        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        self.end_headers()
        self.wfile.write(body)
```

Giải thích: SENSOR_DATA lưu hai giá trị cảm biến. do_GET() xử lý request GET; urlsplit() lấy đường dẫn request. Khi client gọi /api/sensor, json.dumps() chuyển dữ liệu sang JSON, encode() chuyển sang byte để truyền qua mạng.

send_response(200) báo thành công. Các header xác định kiểu nội dung, số byte phản hồi và yêu cầu không dùng dữ liệu cũ từ bộ nhớ đệm. end_headers() kết thúc phần header; wfile.write() gửi JSON tới client. Đường dẫn không tồn tại trả HTTP 404.

Khởi tạo và chạy server

```
try:
    with ThreadingHTTPServer((args.host, args.port), SensorHandler) as server:
        print(f"Server: http://{args.host}:{args.port}", flush=True)
        print("API: GET /api/sensor -> temperature=23, humidity=33", flush=True)
        print("Nhan Ctrl+C de dung server.", flush=True)
        server.serve_forever()
```

ThreadingHTTPServer lắng nghe kết nối tại host/cổng đã cấu hình và xử lý nhiều request. serve_forever() giữ server hoạt động đến khi người dùng nhấn Ctrl+C. Mặc định server chỉ phục vụ trên máy cục bộ (127.0.0.1).

Server còn trả client.html tại đường dẫn / để client trình duyệt có thể gửi request tới cùng server. Do đó người dùng mở trang qua HTTP thay vì mở file HTML trực tiếp.

5. Mã nguồn chính của client

File client.py. Client sử dụng urllib.request để gửi GET, đọc phản hồi và chuyển chuỗi JSON thành dữ liệu Python.

```
import argparse
import json
import math
from urllib.error import HTTPError, URLError
from urllib.request import Request, urlopen


def read_sensor(url):
    request = Request(url, method="GET", headers={"Accept": "application/json"})
    with urlopen(request, timeout=10) as response:
        status = response.status
        data = json.loads(response.read().decode("utf-8"))
    if not isinstance(data, dict):
        raise ValueError("Du lieu JSON phai la mot object.")
    for field in ("temperature", "humidity"):
        value = data.get(field)
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise ValueError(f"Truong {field} phai la so.")
        if not math.isfinite(value):
            raise ValueError(f"Truong {field} phai la so huu han.")
    if not 0 <= data["humidity"] <= 100:
        raise ValueError("Do am phai nam trong khoang 0 den 100%.")
    return status, data


def main():
    parser = argparse.ArgumentParser(description="Client HTTP GET - Bai 1")
    parser.add_argument("--url", default="http://127.0.0.1:8000/api/sensor")
    args = parser.parse_args()
    try:
        print(f"Request: GET {args.url}")
        status, data = read_sensor(args.url)
        print(f"HTTP status: {status} OK")
        print("JSON:", json.dumps(data, ensure_ascii=False))
        print(f"Nhiet do (temperature): {data['temperature']:g} do C")
        print(f"Do am (humidity): {data['humidity']:g}%")
    except HTTPError as exc:
        parser.exit(1, f"Loi HTTP: {exc.code}\n")
    except (URLError, TimeoutError, OSError) as exc:
        parser.exit(1, f"Loi ket noi: {exc}\n")
    except (ValueError, UnicodeError) as exc:
        parser.exit(1, f"Loi du lieu: {exc}\n")

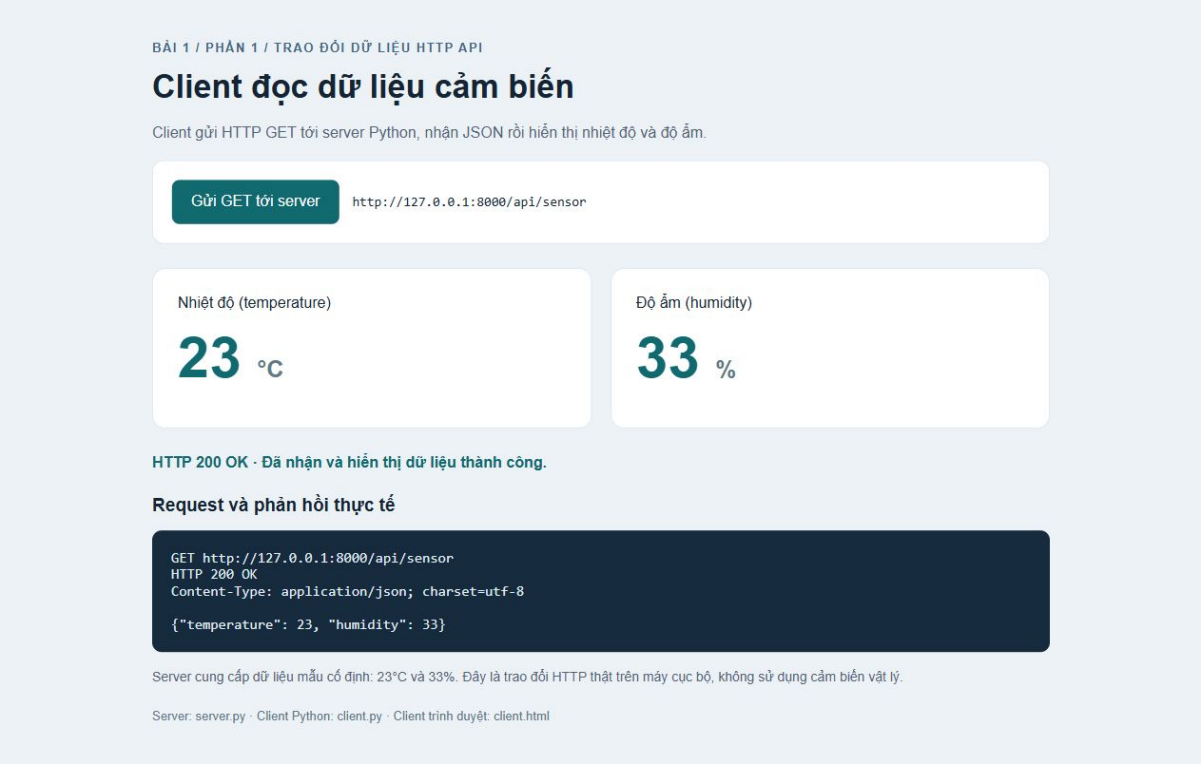

if __name__ == "__main__":
    main()
```

Giải thích: Request(..., method="GET") chỉ định phương thức GET. urlopen(..., timeout=10) mở kết nối với thời gian chờ tối đa 10 giây. response.read() đọc byte, decode("utf-8") giải mã và json.loads() phân tích JSON.

Client kiểm tra hai trường là số hợp lệ, độ ẩm nằm trong khoảng 0-100%, sau đó hiển thị với đơn vị. Các nhánh except báo lỗi HTTP, lỗi kết nối và lỗi dữ liệu; khi lỗi, client kết thúc với mã 1.

6. Ảnh thực hiện chương trình và kết quả

Server và hai client đã được chạy, nhận đúng nhiệt độ 23°C và độ ẩm 33%.



Hình 1. Client trình duyệt gửi GET thật; phản hồi 200 OK và JSON chứa temperature=23, humidity=33.



Hình 2. Ảnh chụp trang hiển thị đầu ra đã lưu từ lần chạy client Python thực tế và kết quả kiểm tra; không phải ảnh cửa sổ terminal.

Kết luận: Đã hoàn thành ba yêu cầu của Phần 1: server có dữ liệu cảm biến; client gửi GET nhận dữ liệu; client hiển thị kết quả. Kiểm tra thêm đường dẫn sai trả 404 và client báo lỗi đúng.

Nguồn yêu cầu: Bài 1. Trao đổi dữ liệu sử dụng HTTP API (VI).pdf, trang 1, mục Phần 1. Mã nguồn đầy đủ và hướng dẫn chạy đi kèm trong file ZIP.